# Curso Avançado

# Capítulo 13 - Operação nos modos Manual e Automático

Definir quem solicita cada etapa sem retirar as verificações que protegem o percurso.

| | |
|---|---|
| Introdução | Nos dois modos, as rotas precisam respeitar ocupação, reservas, posição dos desvios e validade das informações. A diferença principal está em quem solicita o avanço da operação: o operador ou a rotina automática. |
| Objetivo | Organizar a seleção de modo, a origem dos pedidos e a transição controlada entre Manual e Automático. |
| Meta de aprendizagem | Testar um coordenador comum aos dois modos e impedir que a troca de modo libere recursos, contorne o intertravamento ou provoque uma partida inesperada. |

## 1. Manual também precisa de regras

No modo Manual, o operador escolhe a operação e solicita seus comandos. Isso não significa comandar qualquer desvio ou iniciar qualquer movimento. O coordenador continua verificando as condições e pode recusar um pedido incompatível.

No modo Automático, a rotina solicita etapas conforme o plano e os eventos reconhecidos. As mesmas verificações precisam ser aplicadas. A automação não deve possuir um caminho especial que ignore as condições exigidas do operador.

Neste capítulo, os dois modos são simulados no Monitor Serial. Não há comando de tração ou servo. Uma preparação e uma passagem representam o funcionamento básico de uma rota didática.

## 2. Modo de operação e modo de tração

Manual/Automático e Analógico/DCC são escolhas diferentes. O primeiro par define a origem das solicitações; o segundo define a forma de tração. Selecionar Automático não transforma um circuito analógico em DCC.

## 3. Um coordenador para todas as solicitações

| Responsabilidade | Regra comum aos dois modos |
|---|---|
| Reserva | Conceder somente recursos compatíveis e disponíveis. |
| Desvios | Preparar e confirmar posições antes de autorizar passagem. |
| Ocupação | Manter recursos protegidos enquanto a composição puder estar sobre eles. |
| Tração | Enviar somente solicitações permitidas pela operação e pelos perfis. |
| Comunicação | Exigir informações atuais dos equipamentos envolvidos. |
| Falhas | Bloquear a progressão, pedir parada e preservar reservas. |

O painel, o tablet e a rotina automática devem encaminhar pedidos ao mesmo conjunto de decisões. A função que executa o comando físico só recebe o pedido depois dessas verificações.

## 4. Prioridade da emergência

O pedido de emergência precisa ser aceito independentemente do modo. Uma falha ou emergência não deve ser interpretada como ordem para trocar automaticamente de modo e continuar a operação.

O exemplo bloqueia a sequência e mantém a reserva existente. A ação elétrica real de parada segue o projeto de proteção. O estado bloqueado permanece até a inspeção e recuperação previstas.

## 5. Troca de modo com situação conferida

Para este exercício, a troca só é permitida com composição parada, recursos livres e sem operação preparada ou ativa. Essa regra simplifica o estudo e torna a transição explícita.

Em um projeto que permita troca com trem parado dentro de recursos reservados, as reservas precisam ser transferidas ao novo responsável sem interrupção de proteção. Isso exige uma lógica adicional e não está implementado aqui.

Alterar o modo não modifica a posição dos desvios, libera uma rota nem retoma a tração. O painel deve mostrar o modo confirmado pelo coordenador, e não apenas a opção solicitada pelo operador.

## 6. O que muda no exercício

| Etapa | Manual | Automático |
|---|---|---|
| Preparação | P solicita reserva e preparação. | P inicia o ciclo autorizado pelo operador. |
| Conferência | K representa inspeção das agulhas. | K permanece necessário neste exemplo. |
| Autorização | G solicita início da passagem. | Rotina solicita início após K. |
| Eventos de passagem | A e B representam entrada e saída. | Os mesmos eventos são necessários. |
| Liberação | L confirma saída completa. | L continua manual por falta de detecção de saída completa. |

O modo Automático do exemplo é parcial: a conferência dos desvios e a saída completa ainda dependem do operador. Um sistema totalmente automático só poderá substituir essas etapas quando houver informações confiáveis para fazê-lo.

## 7. Seleção e nova partida

O modo confirmado não autoriza um novo ciclo sozinho. Mesmo em Automático, este exemplo exige P para iniciar cada operação. Depois de liberar o percurso, a rotina permanece parada até outra solicitação.

Essa escolha evita um reinício inesperado durante os testes. Uma operação contínua futura precisa prever início explícito, cancelamento, fim de ciclo e limites definidos para a sequência de rotas.

## 8. Comandos do exercício

| Comando | Função |
|---|---|
| I | Inspeção inicial ou recuperação: trem parado e recursos livres. |
| M / U | Solicitar Manual / Automático somente em estado livre. |
| P / K / G | Preparar / confirmar agulhas / solicitar passagem no Manual. |
| A / B / L | Entrada / saída / conferência de saída completa e liberação. |
| X | Emergência em qualquer modo. |

M e U recusados durante a operação não alteram o modo nem a posição física. A conclusão da passagem continua sob o modo que estava confirmado.

## 9. Código em C++ - coordenador e seleção de modo

Reúna as duas partes em um sketch e use 115200 baud. Os estados representam somente a simulação. As conferências de I, K e L são manuais.

```
enum class Mode { MANUAL, AUTOMATIC };
enum class State { BLOCKED, IDLE, PREPARING, READY,
                   RUNNING, OCCUPIED, WAIT_CLEAR };
Mode mode = Mode::MANUAL;
State state = State::BLOCKED;
bool reserved = false;
unsigned long changedAt = 0;
const unsigned long timeoutMs = 60000UL;

void enterState(State next) {
  state = next;
  changedAt = millis();
}
void blockOperation(const char* reason) {
  enterState(State::BLOCKED);
  Serial.println("SIMULATION: stop request. Reservations retained.");
  Serial.println(reason);
}
void requestMode(Mode next) {
  if (state != State::IDLE || reserved) {
    Serial.println("Mode change denied: operation not clear."); return;
  }
  // Operator confirms train stopped and resources clear.
  mode = next;
  Serial.println(mode == Mode::MANUAL ? "MANUAL" : "AUTOMATIC");
  Serial.println("No turnout movement or automatic restart.");
}
void requestRun() {
  // Common gate for manual and automatic requests.
  if (state != State::READY || !reserved) {
    blockOperation("Run conditions not satisfied"); return;
  }
  enterState(State::RUNNING);
  Serial.println("SIMULATION: circulation authorized.");
}
void handleCommand(char c) {
  if (c == 'X') { blockOperation("Emergency"); return; }
  if (c == 'M') { requestMode(Mode::MANUAL); return; }
  if (c == 'U') { requestMode(Mode::AUTOMATIC); return; }
  if (c == 'I' && state == State::BLOCKED) {
    // Inspection: stopped train, resources clear and reconciled.
    reserved = false;
    enterState(State::IDLE);
    Serial.println("Inspected. Await a new preparation request.");
  } else if (c == 'P' && state == State::IDLE && !reserved) {
    reserved = true;
    enterState(State::PREPARING);
    Serial.println("Path reserved. Prepare and inspect turnouts.");
```

## 10. Código em C++ - avanço da operação

```
  } else if (c == 'K' && state == State::PREPARING) {
    // Operator confirms physical turnout positions.
    enterState(State::READY);
    Serial.println("Path checked. READY.");
  } else if (c == 'G' && mode == Mode::MANUAL) {
    requestRun();
  } else if (c == 'G' && mode == Mode::AUTOMATIC) {
    Serial.println("Manual run request denied in AUTOMATIC.");
  } else if (c == 'A' && state == State::RUNNING) {
    enterState(State::OCCUPIED);
    Serial.println("Entry accepted. Reservation kept.");
  } else if (c == 'B' && state == State::OCCUPIED) {
    enterState(State::WAIT_CLEAR);
    Serial.println("SIMULATION: stop request. Await complete exit.");
  } else if (c == 'L' && state == State::WAIT_CLEAR) {
    // Operator confirms stopped train and all resources clear.
    reserved = false;
    enterState(State::IDLE);
    Serial.println("Resources released. No new cycle started.");
  } else {
    blockOperation("Unexpected command or event");
  }
}
void runAutomaticLogic() {
  if (mode == Mode::AUTOMATIC && state == State::READY)
    requestRun();
}
bool needsTimeout() {
  return state == State::PREPARING || state == State::READY ||
         state == State::RUNNING || state == State::OCCUPIED ||
         state == State::WAIT_CLEAR;
}
void setup() {
  Serial.begin(115200);
  blockOperation("Startup: MANUAL selected, inspection required.");
}
void loop() {
  if (needsTimeout() && millis() - changedAt >= timeoutMs)
    blockOperation("Stage timeout");
  while (Serial.available() > 0) {
    char c = Serial.read();
    if (c != '\n' && c != '\r' && c != ' ') handleCommand(c);
  }
  runAutomaticLogic();
}
```

Em Automático, após K e o processamento das entradas, a rotina chama a mesma requestRun usada no Manual. Portanto, K conclui a preparação e permite o início automático no ciclo que foi solicitado por P.

O prazo de 60 segundos é didático e reinicia a cada etapa. I mantém o modo vigente na recuperação, mas não inicia movimento. Após reinício da placa, o modo volta a Manual e a operação começa bloqueada.

## 11. Roteiro de testes

| Teste | Resultado esperado |
|---|---|
| I M P K G A B L | Ciclo Manual. G é necessário para iniciar passagem. |
| I U P K; depois A B L | Ciclo Automático. Após K, rotina solicita passagem sem G. |
| M ou U durante preparação | Troca recusada. Reserva e modo vigente mantidos. |
| U após ciclo Manual liberado | Automático selecionado. Nenhuma partida antes de novo P. |
| X durante qualquer ciclo | Pedido de parada e bloqueio. Reserva mantida. |
| I após falha, condições conferidas | Retorna a IDLE sem iniciar operação. |
| Reiniciar o controlador | Manual selecionado e estado bloqueado. |
| Esperar mais de 60 s em etapa ativa | Falha por prazo excedido. Sem liberação automática. |

Para testar I novamente, entre primeiro em bloqueio com X e faça as inspeções exigidas. Não envie I se houver trem nos recursos protegidos. A liberação por recuperação pressupõe todos os recursos conferidos e livres.

## 12. Integração com painel e modos de tração

Mostre modo solicitado, modo confirmado e motivo de recusa. A chave ou o botão representa o pedido; o coordenador confirma a mudança quando suas condições estão satisfeitas. Durante a transição, preserve a proteção dos recursos.

As regras de Analógico/DCC devem ser tratadas separadamente e integradas ao projeto vigente. A mudança de forma de tração exige suas próprias verificações e não deve ser produzida por M ou U deste exercício.

Com várias interfaces, defina autoridade para selecionar modo e iniciar ciclos. A perda de conexão do tablet não deve transferir automaticamente o controle nem retomar um pedido antigo.

## 13. Exercício e registro no Diário de Bordo

Descreva uma operação Manual e sua versão Automática. Liste quais solicitações serão produzidas pelo operador e quais pela rotina. Mantenha explícitas as confirmações físicas e as condições de liberação.

Teste troca durante reserva, emergência e recuperação. Registre modo solicitado, modo confirmado, etapa e resultado. A meta é mudar a origem dos pedidos mantendo as mesmas verificações de percurso.